

2

Basic Thread Synchronization

In this chapter, we will cover the following topics:

- Synchronizing a method
- Using conditions in synchronized code
- Synchronizing a block of code with a lock
- Synchronizing data access with read/write locks
- Using multiple conditions in a lock
- Advanced locking with the StampedLock class

Introduction

One of the most common situations in concurrent programming occurs when more than one execution thread shares a resource. In a concurrent application, it is normal for multiple threads to read or write the same data structure or have access to the same file or database connection. These shared resources can provoke error situations or data inconsistency, and we have to implement mechanisms to avoid these errors. These situations are called **race conditions** and they occur when different threads have access to the same shared resource at the same time. Therefore, the final result depends on the order of the execution of threads, and most of the time, it is incorrect. You can also have problems with change visibility. So if a thread changes the value of a shared variable, the changes would only be written in the local cache of that thread; other threads will not have access to the change (they will only be able to see the old value).

The solution for these problems lies in the concept of **critical section**. A critical section is a block of code that accesses a shared resource and can't be executed by more than one thread at the same time.

To help programmers implement critical sections, Java (and almost all programming languages) offers synchronization mechanisms. When a thread wants access to a critical section, it uses one of these synchronization mechanisms to find out whether there is any other thread executing the critical section. If not, the thread enters the critical section. If yes, the thread is suspended by the synchronization mechanism until the thread that is currently executing the critical section ends it. When more than one thread is waiting for a thread to finish the execution of a critical section, JVM chooses one of them and the rest wait for their turn. This chapter presents a number of recipes that will teach you how to use the two basic synchronization mechanisms offered by the Java language:

- The `synchronized` keyword
- The `Lock` interface and its implementations

Synchronizing a method

In this recipe, you will learn how to use one of the most basic methods of synchronization in Java, that is, the use of the `synchronized` keyword to control concurrent access to a method or a block of code. All the `synchronized` sentences (used on methods or blocks of code) use an object reference. Only one thread can execute a method or block of code protected by the same object reference.

When you use the `synchronized` keyword with a method, the object reference is implicit. When you use the `synchronized` keyword in one or more methods of an object, only one execution thread will have access to all these methods. If another thread tries to access any method declared with the `synchronized` keyword of the same object, it will be suspended until the first thread finishes the execution of the method. In other words, every method declared with the `synchronized` keyword is a critical section, and Java only allows the execution of one of the critical sections of an object at a time. In this case, the object reference used is the `own` object, represented by the `this` keyword. Static methods have a different behavior. Only one execution thread will have access to one of the static methods declared with the `synchronized` keyword, but a different thread can access other non-static methods of an object of that class. You have to be very careful with this point because two threads can access two different `synchronized` methods if one is static and the other is not. If both methods change the same data, you can have data inconsistency errors. In this case, the object reference used is the class object.

When you use the `synchronized` keyword to protect a block of code, you must pass an object reference as a parameter. Normally, you will use the `this` keyword to reference the object that executes the method, but you can use other object references as well. Normally, these objects will be created exclusively for this purpose. You should keep the objects used for synchronization private. For example, if you have two independent attributes in a class shared by multiple threads, you must synchronize access to each variable; however, it wouldn't be a problem if one thread is accessing one of the attributes and the other accessing a different attribute at the same time. Take into account that if you use the `own` object (represented by the `this` keyword), you might interfere with other synchronized code (as mentioned before, the `this` object is used to synchronize the methods marked with the `synchronized` keyword).

In this recipe, you will learn how to use the `synchronized` keyword to implement an application simulating a parking area, with sensors that detect the following: when a car or a motorcycle enters or goes out of the parking area, an object to store the statistics of the vehicles being parked, and a mechanism to control cash flow. We will implement two versions: one without any synchronization mechanisms, where we will see how we obtain incorrect results, and one that works correctly as it uses the two variants of the `synchronized` keyword.

Getting ready

The example of this recipe has been implemented using the Eclipse IDE. If you use Eclipse or a different IDE, such as NetBeans, open it and create a new Java project.

How to do it...

Follow these steps to implement the example:

1. First, create the application without using any synchronization mechanism. Create a class named `ParkingCash` with an internal constant and an attribute to store the total amount of money earned by providing this parking service:

```
public class ParkingCash {
    private static final int cost=2;
    private long cash;

    public ParkingCash() {
        cash=0;
    }
}
```

2. Implement a method named `vehiclePay()` that will be called when a vehicle (a car or motorcycle) leaves the parking area. It will increase the cash attribute:

```
public void vehiclePay() {
    cash+=cost;
}
```

3. Finally, implement a method named `close()` that will write the value of the cash attribute in the console and reinitialize it to zero:

```
public void close() {
    System.out.printf("Closing accounting");
    long totalAmmount;
    totalAmmount=cash;
    cash=0;
    System.out.printf("The total amount is : %d",
        totalAmmount);
}
}
```

4. Create a class named `ParkingStats` with three private attributes and the constructor that will initialize them:

```
public class ParkingStats {
    private long numberCars;
    private long numberMotorcycles;
    private ParkingCash cash;

    public ParkingStats(ParkingCash cash) {
        numberCars = 0;
        numberMotorcycles = 0;
        this.cash = cash;
    }
}
```

5. Then, implement the methods that will be executed when a car or motorcycle enters or leaves the parking area. When a vehicle leaves the parking area, cash should be incremented:

```
public void carComeIn() {
    numberCars++;
}

public void carGoOut() {
    numberCars--;
    cash.vehiclePay();
}
```

```
public void motoComeIn() {
    numberMotorcycles++;
}

public void motoGoOut() {
    numberMotorcycles--;
    cash.vehiclePay();
}
```

6. Finally, implement two methods to obtain the number of cars and motorcycles in the parking area, respectively.
7. Create a class named `Sensor` that will simulate the movement of vehicles in the parking area. It implements the `Runnable` interface and has a `ParkingStats` attribute, which will be initialized in the constructor:

```
public class Sensor implements Runnable {

    private ParkingStats stats;

    public Sensor(ParkingStats stats) {
        this.stats = stats;
    }
}
```

8. Implement the `run()` method. In this method, simulate that two cars and a motorcycle arrive in and then leave the parking area. Every sensor will perform this action 10 times:

```
@Override
public void run() {
    for (int i = 0; i < 10; i++) {
        stats.carComeIn();
        stats.carComeIn();
        try {
            TimeUnit.MILLISECONDS.sleep(50);
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
        stats.motoComeIn();
        try {
            TimeUnit.MILLISECONDS.sleep(50);
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }
}
```

```
        stats.motoGoOut();
        stats.carGoOut();
        stats.carGoOut();
    }
}
```

9. Finally, implement the main method. Create a class named `Main` with the `main()` method. It needs `ParkingCash` and `ParkingStats` objects to manage parking:

```
public class Main {

    public static void main(String[] args) {

        ParkingCash cash = new ParkingCash();
        ParkingStats stats = new ParkingStats(cash);

        System.out.printf("Parking Simulator\n");
```

10. Then, create the `Sensor` tasks. Use the `availableProcessors()` method (that returns the number of available processors to the JVM, which normally is equal to the number of cores in the processor) to calculate the number of sensors our parking area will have. Create the corresponding `Thread` objects and store them in an array:

```
int numberSensors=2 * Runtime.getRuntime()
                        .availableProcessors();
Thread threads[]=new Thread[numberSensors];
for (int i = 0; i<numberSensors; i++) {
    Sensor sensor=new Sensor(stats);
    Thread thread=new Thread(sensor);
    thread.start();
    threads[i]=thread;
}
```

11. Then wait for the finalization of the threads using the `join()` method:

```
for (int i=0; i<numberSensors; i++) {
    try {
        threads[i].join();
    } catch (InterruptedException e) {
        e.printStackTrace();
    }
}
```

12. Finally, write the statistics of Parking:

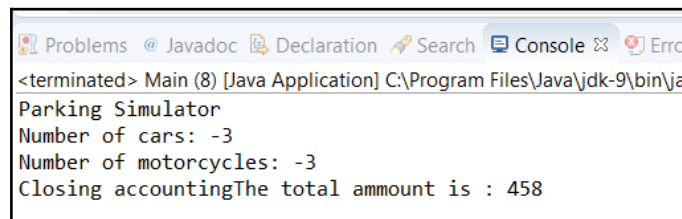
```
        System.out.printf("Number of cars: %d\n",
                           stats.getNumberCars());
        System.out.printf("Number of motorcycles: %d\n",
                           stats.getNumberMotorcycles());
        cash.close();
    }
}
```

In our case, we executed the example in a four-core processor, so we will have eight `Sensor` tasks. Each task performs 10 iterations, and in each iteration, three vehicles enter the parking area and the same three vehicles go out. Therefore, each `Sensor` task will simulate 30 vehicles.

If everything goes well, the final stats will show the following:

- There are no cars in the parking area, which means that all the vehicles that came into the parking area have moved out
- Eight `Sensor` tasks were executed, where each task simulated 30 vehicles and each vehicle was charged 2 dollars each; therefore, the total amount of cash earned was 480 dollars

When you execute this example, each time you will obtain different results, and most of them will be incorrect. The following screenshot shows an example:



We had race conditions, and the different shared variables accessed by all the threads gave incorrect results. Let's modify the previous code using the `synchronized` keyword to solve these problems:

1. First, add the `synchronized` keyword to the `vehiclePay()` method of the `ParkingCash` class:

```
public synchronized void vehiclePay() {
    cash+=cost;
}
```

2. Then, add a synchronized block of code using the `this` keyword to the `close()` method:

```
public void close() {
    System.out.printf("Closing accounting");
    long totalAmmount;
    synchronized (this) {
        totalAmmount=cash;
        cash=0;
    }
    System.out.printf("The total amount is : %d",totalAmmount);
}
```

3. Now add two new attributes to the `ParkingStats` class and initialize them in the constructor of the class:

```
private final Object controlCars, controlMotorcycles;
public ParkingStats (ParkingCash cash) {
    numberCars=0;
    numberMotorcycles=0;
    controlCars=new Object();
    controlMotorcycles=new Object();
    this.cash=cash;
}
```

4. Finally, modify the methods that increment and decrement the number of cars and motorcycles, including the `synchronized` keyword. The `numberCars` attribute will be protected by the `controlCars` object, and the `numberMotorcycles` attribute will be protected by the `controlMotorcycles` object. You must also synchronize the `getNumberCars()` and `getNumberMotorcycles()` methods with the associated reference object:

```
public void carComeIn() {
    synchronized (controlCars) {
        numberCars++;
    }
}

public void carGoOut() {
    synchronized (controlCars) {
        numberCars--;
    }
    cash.vehiclePay();
}
```



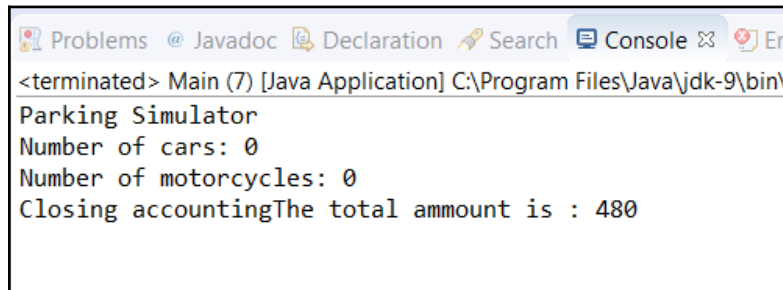
```
public void motoComeIn() {
    synchronized (controlMotorcycles) {
        numberMotorcycles++;
    }
}

public void motoGoOut() {
    synchronized (controlMotorcycles) {
        numberMotorcycles--;
    }
    cash.vehiclePay();
}
```

5. Execute the example now and see the difference when compared to the previous version.

How it works...

The following screenshot shows the output of the new version of the example. No matter how many times you execute it, you will always obtain the correct result:



Let's see the different uses of the `synchronized` keyword in the example:

- First, we protected the `vehiclePay()` method. If two or more `Sensor` tasks call this method at the same time, only one will execute it and the rest will wait for their turn; therefore, the final amount will always be correct.
- We used two different objects to control access to the car and motorcycle counters. This way, one `Sensor` task can modify the `numberCars` attribute and another `Sensor` task can modify the `numberMotorcycles` attribute at the same time; however, no two `Sensor` tasks will be able to modify the same attribute at the same time, so the final value of the counters will always be correct.

Finally, we also synchronized the `getNumberCars()` and `getNumberMotorcycles()` methods. Using the `synchronized` keyword, we can guarantee correct access to shared data in concurrent applications.

As mentioned at the introduction of this recipe, only one thread can access the methods of an object that uses the `synchronized` keyword in their declaration. If thread (A) is executing a `synchronized` method and thread (B) wants to execute another `synchronized` method of the same object, it will be blocked until thread (A) is finished. But if thread (B) has access to different objects of the same class, none of them will be blocked.

When you use the `synchronized` keyword to protect a block of code, you use an object as a parameter. JVM guarantees that only one thread can have access to all the blocks of code protected with this object (note that we always talk about objects, not classes).

We used the `TimeUnit` class as well. The `TimeUnit` class is an enumeration with the following constants: `DAYS`, `HOURS`, `MICROSECONDS`, `MILLISECONDS`, `MINUTES`, `NANOSECONDS`, and `SECONDS`. These indicate the units of time we pass to the `sleep` method. In our case, we let the thread sleep for 50 milliseconds.

There's more...

The `synchronized` keyword penalizes the performance of the application, so you must only use it on methods that modify shared data in a concurrent environment. If you have multiple threads calling a `synchronized` method, only one will execute them at a time while the others will remain waiting. If the operation doesn't use the `synchronized` keyword, all the threads can execute the operation at the same time, reducing the total execution time. If you know that a method will not be called by more than one thread, don't use the `synchronized` keyword. Anyway, if the class is designed for multithreading access, it should always be correct. You must promote correctness over performance. Also, you should include documentation in methods and classes in relation to their thread safety.

You can use recursive calls with `synchronized` methods. As the thread has access to the `synchronized` methods of an object, you can call other `synchronized` methods of that object, including the method that is being executed. It won't have to get access to the `synchronized` methods again.

We can use the `synchronized` keyword to protect access to a block of code instead of an entire method. We should use the `synchronized` keyword in this way to protect access to shared data, leaving the rest of the operations out of this block and obtaining better performance of the application. The objective is to have the critical section (the block of code that can be accessed only by one thread at a time) as short as possible. Also, avoid calling blocking operations (for example, I/O operations) inside a critical section. We have used the `synchronized` keyword to protect access to the instruction that updates the number of persons in the building, leaving out the long operations of the block that don't use shared data. When you use the `synchronized` keyword in this way, you must pass an object reference as a parameter. Only one thread can access the `synchronized` code (blocks or methods) of this object. Normally, we will use the `this` keyword to reference the object that is executing the method:

```
synchronized (this) {  
    // Java code  
}
```

See also

- The *Using conditions in synchronized code* recipe in this chapter

Using conditions in synchronized code

A classic problem in concurrent programming is the producer-consumer problem. We have a data buffer, one or more producers of data that save it in the buffer, and one or more consumers of data that take it from the buffer.

As the buffer is a shared data structure, we have to control access to it using a synchronization mechanism, such as the `synchronized` keyword, but here we have more limitations. A producer can't save data in the buffer if it's full, and a consumer can't take data from the buffer if it's empty.

For these types of situations, Java provides the `wait()`, `notify()`, and `notifyAll()` methods implemented in the `Object` class. A thread can call the `wait()` method inside a synchronized block of code. If it calls the `wait()` method outside a synchronized block of code, JVM throws an `IllegalMonitorStateException` exception. When the thread calls the `wait()` method, JVM puts the thread to sleep and releases the object that controls the synchronized block of code that it's executing and allows other threads to execute other blocks of synchronized code protected by this object. To wake up the thread, you must call the `notify()` or `notifyAll()` methods inside a block of code protected by the same object.

In this recipe, you will learn how to implement the producer-consumer problem using the `synchronized` keyword and the `wait()`, `notify()`, and `notifyAll()` methods.

Getting ready

The example of this recipe has been implemented using the Eclipse IDE. If you use Eclipse or a different IDE, such as NetBeans, open it and create a new Java project.

How to do it...

Follow these steps to implement the example:

1. Create a class named `EventStorage`. It has two attributes, namely an `int` attribute called `maxSize` and a `List<Date>` attribute called `storage`:

```
public class EventStorage {  
  
    private int maxSize;  
    private Queue<Date> storage;  
  
}
```

2. Implement the constructor of the class that initializes the attributes of the class:

```
public EventStorage() {  
    maxSize=10;  
    storage=new LinkedList<>();  
}
```

3. Implement the synchronized method `set()` to store an event in `storage`. First, check whether `storage` is full or not. If it's full, it calls the `wait()` method until it has empty space. At the end of the method, we call the `notify()` method to wake up all the threads that are sleeping in the `wait()` method. In this case, we will ignore `InterruptedException`. In a real implementation, you must think what treatment you must give to them. You can rethrow or transform them into a different type of exception of the application:

```
public synchronized void set(){
    while (storage.size()==maxSize){
        try {
            wait();
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }
    storage.offer(new Date());
    System.out.printf("Set: %d",storage.size());
    notify();
}
```

4. Implement the synchronized method `get()` to get an event for `storage` purposes. First, check whether `storage` has events or not. If it has no events, it calls the `wait()` method until it is given some events. At the end of the method, we call the `notifyAll()` method to wake up all the threads that are sleeping in the `wait()` method. In this case, we will ignore `InterruptedException`. In a real implementation, you must think what treatment you must give to them. You can rethrow or transform them into a different type of exception of the application:

```
public synchronized void get(){
    while (storage.size()==0){
        try {
            wait();
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }
    String element=storage.poll().toString();
    System.out.printf("Get: %d: %s\n",storage.size(),element);
    notify();
}
```

5. Create a class named `Producer` and specify that it implements the `Runnable` interface. It will implement the producer of the example:

```
public class Producer implements Runnable {
```

6. Declare an `EventStore` object and implement the constructor of the class that initializes this object:

```
private EventStorage storage;

public Producer(EventStorage storage){
    this.storage=storage;
}
```

7. Implement the `run()` method that calls the `set()` method of the `EventStorage` object 100 times:

```
@Override
public void run() {
    for (int i=0; i<100; i++){
        storage.set();
    }
}
```

8. Create a class named `Consumer` and specify that it implements the `Runnable` interface. It will implement the consumer of the example:

```
public class Consumer implements Runnable {
```

9. Declare an `EventStorage` object and implement the constructor of the class that initializes this object:

```
private EventStorage storage;

public Consumer(EventStorage storage){
    this.storage=storage;
}
```

10. Implement the `run()` method. It calls the `get()` method of the `EventStorage` object 100 times:

```
@Override
public void run() {
    for (int i=0; i<100; i++){
        storage.get();
    }
}
```

11. Create the main class of the example by implementing a class named `Main` and adding the `main()` method to it:

```
public class Main {

    public static void main(String[] args) {
```

12. Create an `EventStorage` object:

```
EventStorage storage=new EventStorage();
```

13. Create a `Producer` object and `Thread` to run it:

```
Producer producer=new Producer(storage);
Thread thread1=new Thread(producer);
```

14. Create a `Consumer` object and `Thread` to run it:

```
Consumer consumer=new Consumer(storage);
Thread thread2=new Thread(consumer);
```

15. Start both the threads:

```
thread2.start();
thread1.start();
```

How it works...

The key to this example is the `set()` and `get()` methods of the `EventStorage` class. First of all, the `set()` method checks whether there is free space in the storage attribute. If it's full, it calls the `wait()` method to wait for free space. When the other thread calls the `notify()` method, this thread wakes up and checks the condition again. The `notify()` method doesn't guarantee that the condition is met. This process is repeated until there is free space in storage and it can generate a new event and store it.

The behavior of the `get()` method is similar. First, it checks whether there are events on the storage attribute. If the `EventStorage` class is empty, it calls the `wait()` method to wait for events. When the other thread calls the `notify()` method, this thread wakes up and checks the condition again until there are some events in storage.



You have to keep checking the conditions and calling the `wait()` method in a `while` loop. You will not be able to continue until the condition is `true`.

If you run this example, you will find that although the producer and consumer are setting and getting events, storage never has the capacity to include more than 10 events.

There's more...

There are other important uses of the `synchronized` keyword. See the *See also* section of this recipes that explain the use of this keyword.

See also

- The *Synchronizing a method* recipe in this chapter

Synchronizing a block of code with a lock

Java provides another mechanism for synchronizing blocks of code. It's a more powerful and flexible mechanism than the `synchronized` keyword. It's based on the `Lock` (of the `java.util.concurrent.locks` package) interface and classes that implement it (as `ReentrantLock`). This mechanism presents some advantages, which are as follows:

- It allows you to structure synchronized blocks in a more flexible way. With the `synchronized` keyword, you only have control over a synchronized block of code in a structured way. However, the `Lock` interface allows you to get more complex structures to implement your critical section.
- The `Lock` interface provides additional functionalities over the `synchronized` keyword. One of the new functionalities is implemented by the `tryLock()` method. This method tries to get control of the lock, and if it can't, because it's used by another thread, it returns `false`. With the `synchronized` keyword, if thread (A) tries to execute a synchronized block of code when thread (B) is executing it, thread (A) is suspended until thread (B) finishes the execution of the synchronized block. With lock, you can execute the `tryLock()` method. This method returns a `Boolean` value indicating whether there is another thread running the code protected by this lock.
- The `ReadWriteLock` interface allows a separation of read and write operations with multiple readers and only one modifier.
- The `Lock` interface offers better performance than the `synchronized` keyword.

The constructor of the `ReentrantLock` class admits a `boolean` parameter named `fair`; this parameter allows you to control its behavior. The `false` value is the default value and it's called the **non-fair mode**. In this mode, if some threads are waiting for a lock and the lock has to select one of these threads to get access to the critical section, it randomly selects anyone of them. The `true` value is called the **fair mode**. In this mode, if some threads are waiting for a lock and the lock has to select one to get access to a critical section, it selects the thread that has been waiting for the longest period of time. Take into account that the behavior explained previously is only used in the `lock()` and `unlock()` methods. As the `tryLock()` method doesn't put the thread to sleep if the `Lock` interface is used, the `fair` attribute doesn't affect its functionality.

In this recipe, you will learn how to use locks to synchronize a block of code and create a critical section using the `Lock` interface and the `ReentrantLock` class that implements it, implementing a program that simulates a print queue. You will also learn how the `fair` parameter affects the behavior of `Lock`.

Getting ready

The example in this recipe has been implemented using the Eclipse IDE. If you use Eclipse or a different IDE, such as NetBeans, open it and create a new Java project.

How to do it...

Follow these steps to implement the example:

1. Create a class named `PrintQueue` that will implement the print queue:

```
public class PrintQueue {
```

2. Declare a `Lock` object and initialize it with a new object of the `ReentrantLock` class in the constructor. The constructor will receive a `Boolean` parameter we will use to specify the fair mode of the `Lock`:

```
    private Lock queueLock;  
    public PrintQueue(boolean fairMode) {  
        queueLock = new ReentrantLock(fairMode);  
    }
```

3. Implement the `printJob()` method. It will receive `Object` as a parameter and it will not return any value:

```
    public void printJob(Object document){
```

4. Inside the `printJob()` method, get control of the `Lock` object by calling the `lock()` method:

```
        queueLock.lock();
```

5. Then, include the following code to simulate the process of printing a document:

```
        try {  
            Long duration=(long) (Math.random()*10000);  
            System.out.println(Thread.currentThread().getName()+ "  
                                PrintQueue: Printing a Job during "+  
                                (duration/1000)+" seconds");  
            Thread.sleep(duration);  
        } catch (InterruptedException e) {  
            e.printStackTrace();  
        }
```

6. Finally, free the control of the `Lock` object with the `unlock()` method:

```
finally {  
    queueLock.unlock();  
}
```

7. Then, repeat the same process again. The `printJob()` method will help you get access to the lock and then free it twice. This strange behavior will allow us to see the difference between fair and non-fair mode in a better way. We include this piece of code in the `printJob()` method:

```
queueLock.lock();  
try {  
    Long duration = (long) (Math.random() * 10000);  
    System.out.printf("%s: PrintQueue: Printing a Job during  
                      %d seconds\n", Thread.currentThread()  
                      .getName(), (duration / 1000));  
    Thread.sleep(duration);  
} catch (InterruptedException e) {  
    e.printStackTrace();  
} finally {  
    queueLock.unlock();  
}
```

8. Create a class named `Job` and specify that it implements the `Runnable` interface:

```
public class Job implements Runnable {
```

9. Declare an object of the `PrintQueue` class and implement the constructor of the class that initializes this object:

```
private PrintQueue printQueue;  
  
public Job(PrintQueue printQueue) {  
    this.printQueue=printQueue;  
}
```

10. Implement the `run()` method. It uses the `PrintQueue` object to send a job to print:

```
@Override
public void run() {
    System.out.printf("%s: Going to print a document\n",
                      Thread.currentThread().getName());
    printQueue.printJob(new Object());
    System.out.printf("%s: The document has been printed\n",
                      Thread.currentThread().getName());
}
```

11. Create the main class of the application by implementing a class named `Main` and adding the `main()` method to it:

```
public class Main {

    public static void main (String args[]){
```

12. We are going to test the `PrintQueue` class using a lock with the fair mode returning both `true` and `false`. We will use an auxiliary method to implement both the tests so the code of the `main()` method is simple:

```
        System.out.printf("Running example with fair-mode =
                           false\n");
        testPrintQueue(false);
        System.out.printf("Running example with fair-mode = true\n");
        testPrintQueue(true);
    }
```

13. Create the `testPrintQueue()` method and create a shared `PrintQueue` object inside it:

```
        private static void testPrintQueue(Boolean fairMode) {
            PrintQueue printQueue=new PrintQueue(fairMode);
```

14. Create 10 `Job` objects and 10 threads to run them:

```
        Thread thread[]=new Thread[10];
        for (int i=0; i<10; i++){
            thread[i]=new Thread(new Job(printQueue),"Thread "+ i);
        }
```

15. Start the 10 threads:

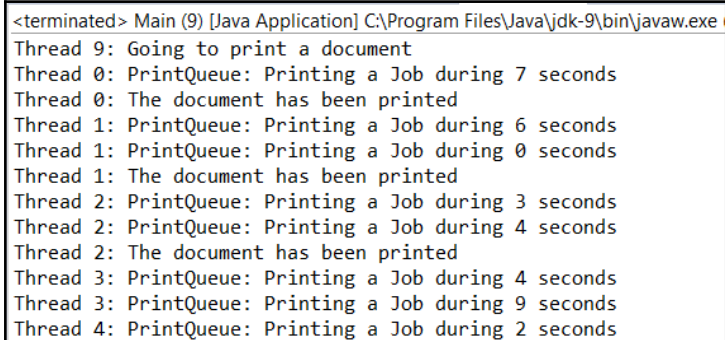
```
for (int i=0; i<10; i++){
    thread[i].start();
}
```

16. Lastly, wait for the finalization of the 10 threads:

```
for (int i=0; i<10; i++) {
    try {
        thread[i].join();
    } catch (InterruptedException e) {
        e.printStackTrace();
    }
}
```

How it works...

In the following screenshot, you can see a part of the output of one execution of this example:

A screenshot of a Java application window titled "<terminated> Main (9) [Java Application] C:\Program Files\Java\jdk-9\bin\javaw.exe". The window displays the output of a multi-threaded program. The output shows 10 threads (0-9) each performing a task: "Thread X: PrintQueue: Printing a Job during Y seconds" followed by "Thread X: The document has been printed". The threads are interleaved, showing concurrent execution. For example, Thread 9 prints first, followed by Thread 0, then Thread 1, and so on, with some threads appearing multiple times in the sequence.

```
<terminated> Main (9) [Java Application] C:\Program Files\Java\jdk-9\bin\javaw.exe
Thread 9: Going to print a document
Thread 0: PrintQueue: Printing a Job during 7 seconds
Thread 0: The document has been printed
Thread 1: PrintQueue: Printing a Job during 6 seconds
Thread 1: PrintQueue: Printing a Job during 0 seconds
Thread 1: The document has been printed
Thread 2: PrintQueue: Printing a Job during 3 seconds
Thread 2: PrintQueue: Printing a Job during 4 seconds
Thread 2: The document has been printed
Thread 3: PrintQueue: Printing a Job during 4 seconds
Thread 3: PrintQueue: Printing a Job during 9 seconds
Thread 4: PrintQueue: Printing a Job during 2 seconds
```

The key to the example is in the `printJob()` method of the `PrintQueue` class. When we want to implement a critical section using locks and guarantee that only one execution thread will run a block of code, we have to create a `ReentrantLock` object. At the beginning of the critical section, we have to get control of the lock using the `lock()` method. When thread (A) calls this method, if no other thread has control of the lock, it gives thread (A) control of the lock and returns immediately to allow the thread to execute the critical section. Otherwise, if there is another, say thread (B), executing the critical section controlled by this lock, the `lock()` method puts thread (A) to sleep until thread (B) finishes the execution of the critical section.

At the end of the critical section, we have to use the `unlock()` method to free the control of the lock and allow other threads to run the critical section. If you don't call the `unlock()` method at the end of the critical section, other threads that are waiting for the lock will wait forever, causing a deadlock situation. If you use try-catch blocks in your critical section, don't forget to put the sentence containing the `unlock()` method inside the `finally` section.

The other topic we tested in this example was fair mode. We had two critical sections in every job. In the previous screenshot, you saw how all the jobs execute the second part immediately after the first one. This is the usual case, but there are exceptions. This occurs when we have non-fair mode, that is to say, we pass a false value to the constructor of the `ReentrantLock` class.

On the contrary, when we establish fair mode by passing the true value to the constructor of the `Lock` class, the behavior is different. The first thread that requests control of the lock is Thread 0, then Thread 1, and so on. While Thread 0 is running the first block of code protected by the lock, we have nine threads waiting to execute the same block of code. When Thread 0 releases the lock, it immediately requests the lock again, so we have 10 threads trying to get the lock. As the fair mode is enabled, the `Lock` interface will choose Thread 1, as it's the thread that has been waiting for more time for the lock. Then, it chooses Thread 2, then Thread 3, and so on. Until all the threads have passed the first block protected by the lock, none of them will execute the second block protected by the lock. Once all the threads have executed the first block of code protected by the lock, then it will be the turn of Thread 0 again, then Thread 1, and so on. The following screenshot shows the difference:

```
<terminated> Main (9) [Java Application] C:\Program Files\Java\jdk-9\bin\javaw.exe
Thread 9: Going to print a document
Thread 1: PrintQueue: Printing a Job during 4 seconds
Thread 2: PrintQueue: Printing a Job during 3 seconds
Thread 3: PrintQueue: Printing a Job during 3 seconds
Thread 4: PrintQueue: Printing a Job during 7 seconds
Thread 5: PrintQueue: Printing a Job during 6 seconds
Thread 6: PrintQueue: Printing a Job during 6 seconds
Thread 7: PrintQueue: Printing a Job during 2 seconds
Thread 8: PrintQueue: Printing a Job during 7 seconds
Thread 9: PrintQueue: Printing a Job during 5 seconds
Thread 0: PrintQueue: Printing a Job during 8 seconds
Thread 0: The document has been printed
Thread 1: PrintQueue: Printing a Job during 5 seconds
Thread 2: PrintQueue: Printing a Job during 9 seconds
Thread 1: The document has been printed
```

There's more...

The `Lock` interface (and the `ReentrantLock` class) includes another method to get control of the lock. It's the `tryLock()` method. The biggest difference with the `lock()` method is that this method, if the thread that uses it can't get control of the `Lock` interface, returns immediately and doesn't put the thread to sleep. It returns the `boolean` value `true` if the thread gets control of the lock and `false` if not. You can also pass a time value and a `TimeUnit` object to indicate the maximum amount of time the thread will wait to get the lock. If the time elapses and the thread doesn't get the lock, the method will return the `false` value. The `TimeUnit` class is an enumeration with the following constants: `DAYS`, `HOURS`, `MICROSECONDS`, `MILLISECONDS`, `MINUTES`, `NANOSECONDS`, and `SECONDS`; these indicate the units of time we pass to a method.



Take into consideration that it is the responsibility of the programmer to take into account the result of this method and act accordingly. If the method returns `false`, it's apparent that your program is unable to execute the critical section. If it does, you probably will have wrong results in your application.

The `ReentrantLock` class also allows the use of recursive calls. When a thread has control of a lock and makes a recursive call, it continues with the control of the lock, so the calling to the `lock()` method will return immediately and the thread will continue with the execution of the recursive call. Moreover, we can also call other methods. You should call the `unlock()` method as many times as you called the `lock()` method in your code.

Avoiding deadlocks

You have to be very careful with the use of locks to avoid **deadlocks**. This situation occurs when two or more threads are blocked while waiting for locks that will never be unlocked. For example, thread (A) locks Lock (X) and thread (B) locks Lock (Y). Now, if thread (A) tries to lock Lock (Y) and thread (B) simultaneously tries to lock Lock (X), both the threads will be blocked indefinitely because they are waiting for locks that will never be liberated. Note that the problem occurs because both threads try to get the locks in the opposite order. The Appendix, *Concurrent Programming Design*, provides some good tips to design concurrent applications adequately and avoid these deadlock problems.

See also

- The *Synchronizing a method* and *Using multiple conditions in a lock* recipes in this chapter
- The *Monitoring a Lock interface* recipe in Chapter 9, *Testing Concurrent Applications*

Synchronizing data access with read/write locks

One of the most significant improvements offered by locks is the `ReadWriteLock` interface and the `ReentrantReadWriteLock` class, the unique class that implements that interface. This class has two locks: one for read operations and one for write operations. There can be more than one thread using read operations simultaneously, but only one thread can use write operations. If a thread is doing a write operation, other threads can't write or read.

In this recipe, you will learn how to use a `ReadWriteLock` interface by implementing a program that uses it to control access to an object that stores the prices of two products.

Getting ready...

You should read the *Synchronizing a block of code with a lock* recipe to better understand this recipe.

How to do it...

Follow these steps to implement the example:

1. Create a class named `PricesInfo` that stores information about the prices of two products:

```
public class PricesInfo {
```

2. Declare two double attributes named `price1` and `price2`:

```
    private double price1;  
    private double price2;
```


3. Declare a `ReadWriteLock` object called `lock`:

```
private ReadWriteLock lock;
```

4. Implement the constructor of the class that initializes the three attributes. For the `lock` attribute, create a new `ReentrantReadWriteLock` object:

```
public PricesInfo() {  
    price1=1.0;  
    price2=2.0;  
    lock=new ReentrantReadWriteLock();  
}
```

5. Implement the `getPrice1()` method that returns the value of the `price1` attribute. It uses the read lock to control access to the value of this attribute:

```
public double getPrice1() {  
    lock.readLock().lock();  
    double value=price1;  
    lock.readLock().unlock();  
    return value;  
}
```

6. Implement the `getPrice2()` method that returns the value of the `price2` attribute. It uses the read lock to control access to the value of this attribute:

```
public double getPrice2() {  
    lock.readLock().lock();  
    double value=price2;  
    lock.readLock().unlock();  
    return value;  
}
```

7. Implement the `setPrices()` method that establishes the values of two attributes. It uses the write lock to control access to them. We are going to make the thread sleep for 5 seconds. This shows that even though it has the write lock, there are no other threads getting the read lock:

```
public void setPrices(double price1, double price2) {  
    lock.writeLock().lock();  
    System.out.printf("%s: PricesInfo: Write Lock Acquired.\n",  
        new Date());  
    try {  
        TimeUnit.SECONDS.sleep(10);  
    } catch (InterruptedException e) {  
        e.printStackTrace();  
    }  
}
```

```
    }
    this.price1=price1;
    this.price2=price2;
    System.out.printf("%s: PricesInfo: Write Lock Released.\n",
        new Date());
    lock.writeLock().unlock();
}
```

8. Create a class named `Reader` and specify that it implements the `Runnable` interface. This class implements a reader of the values of the `PricesInfo` class attribute:

```
public class Reader implements Runnable {
```

9. Declare a `PricesInfo` object and implement the constructor of the class that could initialize this object:

```
    private PricesInfo pricesInfo;

    public Reader (PricesInfo pricesInfo){
        this.pricesInfo=pricesInfo;
    }
}
```

10. Implement the `run()` method for this class. It reads the value of the two prices 10 times:

```
@Override
public void run() {
    for (int i=0; i<20; i++){
        System.out.printf("%s: %s: Price 1: %f\n",new Date(),
            Thread.currentThread().getName(),
            pricesInfo.getPrice1());
        System.out.printf("%s: %s: Price 2: %f\n",new Date(),
            Thread.currentThread().getName(),
            pricesInfo.getPrice2());
    }
}
```

11. Create a class named `Writer` and specify that it implements the `Runnable` interface. This class implements a modifier of the values of the `PricesInfo` class attribute:

```
public class Writer implements Runnable {
```

12. Declare a `PricesInfo` object and implement the constructor of the class that could initialize this object:

```
private PricesInfo pricesInfo;

public Writer(PricesInfo pricesInfo){
    this.pricesInfo=pricesInfo;
}
```

13. Implement the `run()` method. It modifies the value of the two prices that are sleeping for 2 seconds between modifications three times:

```
@Override
public void run() {
    for (int i=0; i<3; i++) {
        System.out.printf("%s: Writer: Attempt to modify the
                           prices.\n", new Date());
        pricesInfo.setPrices(Math.random()*10, Math.random()*8);
        System.out.printf("%s: Writer: Prices have been
                           modified.\n", new Date());

        try {
            Thread.sleep(2);
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }
}
```

14. Implement the main class of the example by creating a class named `Main` and adding the `main()` method to it:

```
public class Main {
    public static void main(String[] args) {
```

15. Create a `PricesInfo` object:

```
PricesInfo pricesInfo=new PricesInfo();
```

16. Create five `Reader` objects and five `Thread` objects to execute them:

```
Reader readers[]=new Reader[5];
Thread threadsReader[]=new Thread[5];

for (int i=0; i<5; i++){
    readers[i]=new Reader(pricesInfo);
    threadsReader[i]=new Thread(readers[i]);
}
```

17. Create a `Writer` object and `Thread` to execute it:

```
Writer writer=new Writer(pricesInfo);
Thread threadWriter=new Thread(writer);
```

18. Start the threads:

```
for (int i=0; i<5; i++){
    threadsReader[i].start();
}
threadWriter.start();
```

How it works...

In the following screenshot, you can see a part of the output of one execution of this example:



```
Mon Aug 29 13:58:01 GMT+01:00 2016: Thread-0: Price 2: 2,000000
Mon Aug 29 13:58:01 GMT+01:00 2016: Thread-2: Price 1: 1,000000
Mon Aug 29 13:58:01 GMT+01:00 2016: Writer: Attempt to modify the prices.
Mon Aug 29 13:58:01 GMT+01:00 2016: Thread-4: Price 1: 1,000000
Mon Aug 29 13:58:01 GMT+01:00 2016: Thread-3: Price 1: 1,000000
Mon Aug 29 13:58:01 GMT+01:00 2016: Thread-1: Price 1: 1,000000
Mon Aug 29 13:58:01 GMT+01:00 2016: PricesInfo: Write Lock Acquired.
Mon Aug 29 13:58:01 GMT+01:00 2016: Thread-2: Price 2: 2,000000
Mon Aug 29 13:58:11 GMT+01:00 2016: PricesInfo: Write Lock Released.
Mon Aug 29 13:58:11 GMT+01:00 2016: Writer: Prices have been modified.
Mon Aug 29 13:58:01 GMT+01:00 2016: Thread-4: Price 2: 5,991554
Mon Aug 29 13:58:11 GMT+01:00 2016: Thread-4: Price 1: 2,259680
Mon Aug 29 13:58:11 GMT+01:00 2016: Thread-4: Price 2: 5,991554
Mon Aug 29 13:58:11 GMT+01:00 2016: Thread-4: Price 1: 2,259680
Mon Aug 29 13:58:11 GMT+01:00 2016: Thread-4: Price 2: 5,991554
Mon Aug 29 13:58:11 GMT+01:00 2016: Thread-4: Price 1: 2,259680
```

While the writer has acquired the write lock, none of the reader tasks can read the data. You can see some messages of the reader tasks after the `Write Lock Acquired` message, but they are instructions that were executed before and not shown yet in the console. Once the writer task has released the lock, reader tasks gain access to the prices information again and show the new prices.

As mentioned previously, the `ReentrantReadWriteLock` class has two locks: one for read operations and one for write operations. The lock used in read operations is obtained with the `readLock()` method declared in the `ReadWriteLock` interface. This lock is an object that implements the `Lock` interface, so we can use the `lock()`, `unlock()`, and `tryLock()` methods. The lock used in write operations is obtained with the `writeLock()` method declared in the `ReadWriteLock` interface. This lock is also an object that implements the `Lock` interface, so we can use the `lock()`, `unlock()`, and `tryLock()` methods. It is the responsibility of the programmer to ensure correct use of these locks, using them for the same purposes for which they were designed. When you get the read lock of a `Lock` interface, you can't modify the value of the variable. Otherwise, you probably will have data errors related to inconsistency.

See also

- The *Synchronizing a block of code with a lock* recipe in this chapter
- The *Monitoring a Lock interface* recipe in Chapter 9, *Testing Concurrent Applications*

Using multiple conditions in a lock

A lock may be associated with one or more conditions. These conditions are declared in the `Condition` interface. The purpose of these conditions is to allow threads to have control of a lock and check whether a condition is `true` or not. If it's `false`, the thread will be suspended until another thread wakes it up. The `Condition` interface provides the mechanisms to suspend a thread and wake up a suspended thread.

A classic problem in concurrent programming is the producer-consumer problem. We have a data buffer, one or more producers of data that save it in the buffer, and one or more consumers of data that take it from the buffer, as explained earlier in this chapter.

In this recipe, you will learn how to implement the producer-consumer problem using locks and conditions.

Getting ready

You should read the *Synchronizing a block of code with a lock* recipe to better understand this recipe.

How to do it...

Follow these steps to implement the example:

1. First, implement a class that will simulate a text file. Create a class named `FileMock` with two attributes: a `String` array named `content` and `int` named `index`. They will store the content of the file and the line of the simulated file that will be retrieved:

```
public class FileMock {  
  
    private String[] content;  
    private int index;
```

2. Implement the constructor of the class that initializes the content of the file with random characters:

```
    public FileMock(int size, int length){  
        content = new String[size];  
        for (int i = 0; i < size; i++){  
            StringBuilder buffer = new StringBuilder(length);  
            for (int j = 0; j < length; j++){  
                int randomCharacter= (int)Math.random()*255;  
                buffer.append((char)randomCharacter);  
            }  
            content[i] = buffer.toString();  
        }  
        index=0;  
    }  
}
```

3. Implement the `hasMoreLines()` method that returns `true` if the file has more lines to process or `false` if you have reached the end of the simulated file:

```
    public boolean hasMoreLines(){  
        return index < content.length;  
    }  
}
```

4. Implement the `getLine()` method that returns the line determined by the index attribute and increases its value:

```
public String getLine(){
    if (this.hasMoreLines()) {
        System.out.println("Mock: " + (content.length-index));
        return content[index++];
    }
    return null;
}
```

5. Now implement a class named `Buffer` that will implement the buffer shared by both the producers and consumers:

```
public class Buffer {
```

6. This class has six attributes:

- A `LinkedList<String>` attribute named `buffer` that will store the shared data. For example:

```
private final LinkedList<String> buffer;
```

- An `int` type named `maxSize` that will store the length of the buffer. For example:

```
private final int maxSize;
```

- A `ReentrantLock` object called `lock` that will control access to the blocks of code that modify the buffer. For example:

```
private final ReentrantLock lock;
```

- Two `Condition` attributes named `lines` and `space`. For example:

```
private final Condition lines;
```

```
private final Condition space;
```

- A `boolean` type called `pendingLines` that will indicate whether there are lines in the buffer. For example:

```
private boolean pendingLines;
```

7. Implement the constructor of the class. It initializes all the attributes described previously:

```
public Buffer(int maxSize) {
    this.maxSize = maxSize;
    buffer = new LinkedList<>();
    lock = new ReentrantLock();
    lines = lock.newCondition();
    space = lock.newCondition();
    pendingLines = true;
}
```

8. Implement the `insert()` method. It receives `String` as a parameter and tries to store it in the buffer. First, it gets control of the lock. When it has this, it checks whether there is empty space in the buffer. If the buffer is full, it calls the `await()` method in the space condition to wait for free space. The thread will be woken up when another thread calls the `signal()` or `signalAll()` method in the space condition. When this happens, the thread stores the line in the buffer and calls the `signalAll()` method over the `lines` condition. As we'll see in a moment, this condition will wake up all the threads that are waiting for lines in the buffer. To make the code easier, we ignore the `InterruptedException` exception. In real cases, you will probably have to process it:

```
public void insert(String line) {
    lock.lock();
    try {
        while (buffer.size() == maxSize) {
            space.await();
        }
        buffer.offer(line);
        System.out.printf("%s: Inserted Line: %d\n",
                        Thread.currentThread().getName(),
                        buffer.size());
        lines.signalAll();
    } catch (InterruptedException e) {
        e.printStackTrace();
    } finally {
        lock.unlock();
    }
}
```


9. Implement the `get()` method. It returns the first string stored in the buffer. First, it gets control of the lock. When this is done, it checks whether there are lines in the buffer. If the buffer is empty, it calls the `await()` method in the `lines` condition to wait for lines in the buffer. This thread will be woken up when another thread calls the `signal()` or `signalAll()` method in the `lines` condition. When this happens, the method gets the first line in the buffer, calls the `signalAll()` method over the `space` condition, and returns `String`:

```
public String get() {
    String line = null;
    lock.lock();
    try {
        while ((buffer.size() == 0) && (hasPendingLines())) {
            lines.await();
        }

        if (hasPendingLines()) {
            line = buffer.poll();
            System.out.printf("%s: Line Readed: %d\n",
                             Thread.currentThread().getName(),
                             buffer.size());
            space.signalAll();
        }
    } catch (InterruptedException e) {
        e.printStackTrace();
    } finally {
        lock.unlock();
    }
    return line;
}
```

10. Implement the `setPendingLines()` method that establishes the value of the `pendingLines` attribute. It will be called by the producer when it has no more lines to produce:

```
public synchronized void setPendingLines(boolean pendingLines) {
    this.pendingLines = pendingLines;
}
```

11. Implement the `hasPendingLines()` method. It returns `true` if there are more lines to be processed or `false` otherwise:

```
public synchronized boolean hasPendingLines() {
    return pendingLines || buffer.size() > 0;
}
```

12. Now it's the turn of the producer. Implement a class named `Producer` and specify that it implements the `Runnable` interface:

```
public class Producer implements Runnable {
```

13. Declare two attributes, namely an object of the `FileMock` class and an object of the `Buffer` class:

```
private FileMock mock;
```

```
private Buffer buffer;
```

14. Implement the constructor of the class that initializes both the attributes:

```
public Producer (FileMock mock, Buffer buffer){  
    this.mock = mock;  
    this.buffer = buffer;  
}
```

15. Implement the `run()` method that reads all the lines created in the `FileMock` object and use the `insert()` method to store them in the buffer. Once this is done, use the `setPendingLines()` method to alert the buffer that it will not generate more lines:

```
@Override  
public void run() {  
    buffer.setPendingLines(true);  
    while (mock.hasMoreLines()){  
        String line = mock.getLine();  
        buffer.insert(line);  
    }  
    buffer.setPendingLines(false);  
}
```

16. Next is the consumer's turn. Implement a class named `Consumer` and specify that it implements the `Runnable` interface:

```
public class Consumer implements Runnable {
```

17. Declare a `Buffer` object and implement the constructor of the class that initializes it:

```
private Buffer buffer;

public Consumer (Buffer buffer) {
    this.buffer = buffer;
}
```

18. Implement the `run()` method. While the buffer has pending lines, it tries to get one line and process it:

```
@Override
public void run() {
    while (buffer.hasPendingLines()) {
        String line = buffer.get();
        processLine(line);
    }
}
```

19. Implement the auxiliary method `processLine()`. It only sleeps for 10 milliseconds to simulate some kind of processing with the line:

```
private void processLine(String line) {
    try {
        Random random = new Random();
        Thread.sleep(random.nextInt(100));
    } catch (InterruptedException e) {
        e.printStackTrace();
    }
}
```

20. Implement the main class of the example by creating a class named `Main` and adding the `main()` method to it.

```
public class Main {

    public static void main(String[] args) {
```

21. Create a `FileMock` object:

```
FileMock mock = new FileMock(100, 10);
```

22. Create a Buffer object:

```
Buffer buffer = new Buffer(20);
```

23. Create a Producer object and Thread to run it:

```
Producer producer = new Producer(mock, buffer);  
Thread producerThread = new Thread(provider, "Producer");
```

24. Create three Consumer objects and three threads to run them:

```
Consumer consumers[] = new Consumer[3];  
Thread consumersThreads[] = new Thread[3];  
  
for (int i=0; i<3; i++){  
    consumers[i] = new Consumer(buffer);  
    consumersThreads[i] = new Thread(consumers[i], "Consumer "+i);  
}
```

25. Start the producer and the three consumers:

```
producerThread.start();  
for (int i = 0; i < 3; i++){  
    consumersThreads[i].start();  
}
```

How it works...

All the `Condition` objects are associated with a lock and are created using the `newCondition()` method declared in the `Lock` interface. Before we can do any operation with a condition, you have to have control of the lock associated with the condition. So operations with conditions must be done in a thread that holds the lock with a call to a `lock()` method of a `Lock` object and then frees it with an `unlock()` method of the same `Lock` object.

When a thread calls the `await()` method of a condition, it automatically frees the control of the lock so that another thread can get it and either begin the execution or another critical section protected by that lock.



When a thread calls the `signal()` or `signalAll()` methods of a condition, one or all of the threads that were waiting for that condition are woken up, but this doesn't guarantee that the condition that made them sleep is now `true`. So you must put the `await()` calls inside a while loop. You can't leave this loop until the condition is `true`. When the condition is `false`, you must call `await()` again.

You must be careful with the use of `await()` and `signal()`. If you call the `await()` method in a condition and never call the `signal()` method in the same condition, the thread will sleep forever.

A thread can be interrupted while it is sleeping, after a call to the `await()` method, so you have to process the `InterruptedException` exception.

There's more...

The `Condition` interface has other versions of the `await()` method, which are as follows:

- `await(long time, TimeUnit unit)`: Here, the thread will sleep until:
 - It's interrupted
 - Another thread calls the `signal()` or `signalAll()` methods in the condition
 - The specified time passes
 - The `TimeUnit` class is an enumeration with the following constants: `DAYS`, `HOURS`, `MICROSECONDS`, `MILLISECONDS`, `MINUTES`, `NANOSECONDS`, and `SECONDS`
- `awaitUninterruptibly()`: The thread will sleep until another thread calls the `signal()` or `signalAll()` methods, which can't be interrupted
- `awaitUntil(Date date)`: The thread will sleep until:
 - It's interrupted
 - Another thread calls the `signal()` or `signalAll()` methods in the condition
 - The specified date arrives

You can use conditions with the `ReadLock` and `WriteLock` locks of a read/write lock.

See also

- The *Synchronizing a block of code with a lock* and *Synchronizing data access with read/write locks* recipes in this chapter

Advanced locking with the StampedLock class

The `StampedLock` class provides a special kind of lock that is different from the ones provided by the `Lock` or `ReadWriteLock` interfaces. In fact, this class doesn't implement these interfaces, but the functionality it provides is very similar.

The first point to note about this kind of lock is that its main purpose is to be a helper class to implement thread-safe components, so its use will not be very common in normal applications.

The most important features of `StampedLock` locks are as follows:

- You can obtain control of the lock in three different modes:
 - **Write:** In this mode, you get exclusive access to the lock. No other thread can have control of the lock in this mode.
 - **Read:** In this mode, you have non-exclusive access to the lock. There can be other threads that have access to the lock in this mode or the Optimistic Read mode.
 - **Optimistic Read:** Here, the thread doesn't have control over the block. Other threads can get control of the lock in write mode. When you get a lock in the Optimistic Read mode and you want to access the shared data protected by it, you will have to check whether you can access them or not using the `validate()` method.
- The `StampedLock` class provides methods to:
 - Acquire control over the lock in one of the previous modes. If the methods `readLock()`, `writeLock()`, `readLockInterruptibly()` are unable to get control of the lock, the current thread is suspended until it gets the lock.

- Acquire control over the lock in one of the previous modes. If the methods (`tryOptimisticRead()`, `tryReadLock()`, `tryWriteLock()`) are unable to get control of the lock, they return a special value to indicate this circumstance.
 - Convert one mode into another, if possible. If not, the methods (`asReadLock()`, `asWriteLock()`, `asReadWriteLock()`) return a special value.
 - Release the lock.
- All these methods return a long value called stamp that we need to use to work with the lock. If a method returns zero, it means it tried to get a lock but it couldn't.
 - A `StampedLock` lock is not a reentrant lock, such as the `Lock` and `ReadWriteLock` interfaces. If you call a method that tries to get the lock again, it may be blocked and you'll get a deadlock.
 - It does not have the notion of ownership. They can be acquired by one thread and released by another.
 - Finally, it doesn't have any policy about the next thread that will get control of the lock.

In this recipe, we will learn how to use the different modes of the `StampedLock` class to protect access to a shared data object. We will use a shared object between three concurrent tasks to test the three access modes with `StampedLock` (write, read, and Optimistic Read).

Getting ready

The example of this recipe has been implemented using the Eclipse IDE. If you use Eclipse or a different IDE, such as NetBeans, open it and create a new Java project.

How to do it...

Follow these steps to implement the example:

1. First, implement the shared data object. Create a class named `Position` with two integer attributes, namely `x` and `y`. You have to include the methods to get and set the values of the attributes. Its code is very simple so it is not included here.

2. Now let's implement the `Writer` task. It implements the `Runnable` interface and it will have two attributes: a `Position` object named `position` and `StampedLock` named `lock`. They will be initialized in the constructor:

```
public class Writer implements Runnable {

    private final Position position;
    private final StampedLock lock;

    public Writer (Position position, StampedLock lock) {
        this.position=position;
        this.lock=lock;
    }
}
```

3. Implement the `run()` method. In a loop that we will repeat 10 times, get the lock in write mode, change the value of the two attributes of the position object, suspend the execution of the thread for a second, release the lock (in the `finally` section of a `try...catch...finally` structure to release the lock in any circumstance), and suspend the thread for a second:

```
@Override
public void run() {

    for (int i=0; i<10; i++) {
        long stamp = lock.writeLock();

        try {
            System.out.printf("Writer: Lock acquired %d\n", stamp);
            position.setX(position.getX()+1);
            position.setY(position.getY()+1);
            TimeUnit.SECONDS.sleep(1);
        } catch (InterruptedException e) {
            e.printStackTrace();
        } finally {
            lock.unlockWrite(stamp);
            System.out.printf("Writer: Lock released %d\n", stamp);
        }

        try {
            TimeUnit.SECONDS.sleep(1);
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }
}
```


4. Then, implement the `Reader` task to read the values of the shared object. Create a class named `Reader` that implements the `Runnable` interface. It will have two attributes: a `Position` object named `position` and a `StampedLock` object named `lock`. They will be initialized in the constructor of the class:

```
public class Reader implements Runnable {  
  
    private final Position position;  
    private final StampedLock lock;  
  
    public Reader (Position position, StampedLock lock) {  
        this.position=position;  
        this.lock=lock;  
    }  
}
```

5. Now implement the `run()` method. In a loop that we will repeat 50 times, get control of the lock in read mode, write the values of the position object in the console, and suspend the thread for 200 milliseconds. Finally, release the lock using the `finally` block of a `try...catch...finally` structure:

```
@Override  
public void run() {  
    for (int i=0; i<50; i++) {  
        long stamp=lock.readLock();  
        try {  
            System.out.printf("Reader: %d - (%d,%d)\n", stamp,  
                               position.getX(), position.getY());  
            TimeUnit.MILLISECONDS.sleep(200);  
        } catch (InterruptedException e) {  
            e.printStackTrace();  
        } finally {  
            lock.unlockRead(stamp);  
            System.out.printf("Reader: %d - Lock released\n", stamp);  
        }  
    }  
}
```

6. Then, implement the `OptimisticReader` task. The class `OptimisticReader` class implements the `Runnable` interface. It will have two attributes: a `Position` object named `position` and a `StampedLock` object named `lock`. They will be initialized in the constructor of the class:

```
public class OptimisticReader implements Runnable {

    private final Position position;
    private final StampedLock lock;

    public OptimisticReader (Position position, StampedLock lock) {
        this.position=position;
        this.lock=lock;
    }
}
```

7. Now implement the `run()` method. First obtain the stamp of the lock in the optimistic read mode using the `tryOptimisticRead()` method. Then, repeat the loop 100 times. In the loop, validate whether you can access data using the `validate()` method. If this method returns true, write the values of the position object in the console. Otherwise, write a message in the console and get another stamp using the `tryOptimisticRead()` method again. Then, suspend the thread for 200 milliseconds:

```
@Override
public void run() {
    long stamp;
    for (int i=0; i<100; i++) {
        try {
            stamp=lock.tryOptimisticRead();
            int x = position.getX();
            int y = position.getY();
            if (lock.validate(stamp)) {
                System.out.printf("OptimisticReader: %d - (%d,%d)\n",
                                stamp,x, y);
            } else {
                System.out.printf("OptimisticReader: %d - Not Free\n",
                                stamp);
            }
            TimeUnit.MILLISECONDS.sleep(200);
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }
}
```

8. Finally, implement the `Main` class with the `main()` method. Create a `Position` and `StampedLock` object, create three threads--one for each task--start the threads, and wait for their finalization:

```
public class Main {

    public static void main(String[] args) {

        Position position=new Position();
        StampedLock lock=new StampedLock();

        Thread threadWriter=new Thread(new Writer(position,lock));
        Thread threadReader=new Thread(new Reader(position, lock));
        Thread threadOptReader=new Thread(new OptimisticReader
                                           (position, lock));

        threadWriter.start();
        threadReader.start();
        threadOptReader.start();

        try {
            threadWriter.join();
            threadReader.join();
            threadOptReader.join();
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }
}
```

How it works...

In this example, we tested the three modes you can use with a stamped lock. In the `Writer` task, we get the lock with the `writeLock()` method (that acquires the lock in write mode). In the `Reader` task, we get the lock with the `readLock()` method (that acquires the lock in read mode). Finally, in the `OptimisticRead` task, first we use `tryOptimisticRead()` and then we use the `validate()` method to check whether we can access data or not.

The first two methods, if they can get control of the lock, wait until they get the lock. The `tryOptimisticRead()` method always returns a value. It will be 0 if we are unable to use the lock and a value different from 0 if we can use it. Remember that in this case, we always need to use the `validate()` method to check whether we can really access the data.

The following screenshot shows part of the output of an execution of the program:

A screenshot of an IDE's console window. The title bar shows 'Problems', 'Javadoc', 'Declaration', 'Search', and 'Console'. The console text shows the execution of a Java application. It starts with '<terminated> Main (10) [Java Application] C:\Program Files\Java\jdk'. The output shows a sequence of messages from different threads: 'Reader: 257 - (0,0)', 'OptmisticReader: 256 - (0,0)', 'OptmisticReader: 256 - (0,0)', 'Writer: Lock aquired 384', 'Reader: 257 - Lock released', 'OptmisticReader: 256 - Not Free', 'OptmisticReader: 0 - Not Free', 'OptmisticReader: 0 - Not Free', 'OptmisticReader: 0 - Not Free', 'Writer: Lock released 384', 'Reader: 513 - (1,1)', 'OptmisticReader: 0 - Not Free', 'Reader: 513 - Lock released', 'Reader: 513 - (1,1)', 'OptmisticReader: 512 - (1,1)', 'Reader: 513 - Lock released', and 'Reader: 513 - (1,1)'.

```
<terminated> Main (10) [Java Application] C:\Program Files\Java\jdk
Reader: 257 - (0,0)
OptmisticReader: 256 - (0,0)
OptmisticReader: 256 - (0,0)
Writer: Lock aquired 384
Reader: 257 - Lock released
OptmisticReader: 256 - Not Free
OptmisticReader: 0 - Not Free
OptmisticReader: 0 - Not Free
OptmisticReader: 0 - Not Free
Writer: Lock released 384
Reader: 513 - (1,1)
OptmisticReader: 0 - Not Free
Reader: 513 - Lock released
Reader: 513 - (1,1)
OptmisticReader: 512 - (1,1)
Reader: 513 - Lock released
Reader: 513 - (1,1)
```

While the `Writer` task has control of the lock, neither `Reader` nor `OptimisticReader` can access the values. The `Reader` task is suspended in the `readLock()` method, while in `OptimisticReader`, the call to the `validate()` method returns `false` and the call to the `tryOptimisticRead()` method returns `0` to indicate that the lock is controlled in write mode by another thread. When the `Writertask` releases the lock, both `Reader` and `OptimisticReader` tasks will be able to access the values of the shared object.

There's more...

The `StampedLock` class has other methods that you should know:

- `tryReadLock()` and `tryReadLock(long time, TimeUnit unit)`: These methods try to acquire the lock in read mode. If they can't, the first version is returned immediately and the second one waits for the amount of time specified in the parameter. These methods also return a stamp that must be checked (`stamp != 0`).

- `tryWriteLock()` and `tryWriteLock(long time, TimeUnit unit)`: These methods try to acquire the lock in write mode. If they can't, the first version is returned immediately and the second one waits for the amount of time specified in the parameter. These methods also return a stamp that must be checked (`stamp != 0`).
- `isReadLocked()` and `isWriteLocked()`: These methods are returned if the lock is currently held in read or write mode, respectively.
- `tryConvertToReadLock(long stamp)`, `tryConvertToWriteLock(long stamp)`, and `tryConvertToOptimisticRead(long stamp)`: These methods try to convert the stamp passed as a parameter to the mode indicated in the name of the method. If they can, they return a new stamp. If not, they return 0.
- `unlock(long stamp)`: This releases the corresponding mode of the lock.

See also

- The *Synchronizing a block of code with a lock* recipe in this chapter